B.Sc. (Data Science)

DS101T : Problem Solving and Python Programming

Unit 2: Introduction to Python

Mrs. Reshma Masurekar
Dr. D. Y. Patil Arts, Commerce and Science College,
Pimpri, Pune 18

- String Manipulation - Accessing String, Basic Operations, String Slices, Function and Methods, Examples.
- Lists-Introduction, accessing list, operations, working with lists, function & methods.
- Tuple-Introduction, accessing tuples, operations working, function & methods, Examples.
- Dictionaries-Introduction, Accessing values in dictionaries, working with dictionaries, properties, function, Examples.

String Manipulation

- In Python, *a string is a sequence of characters enclosed within either single quotes ('...') or doubles quotes ("...").*
- It is *ordered and immutable* built-in data structure, meaning once a string is created, it cannot be modified.
- String manipulation is the process of manipulating and analyzing strings. It includes a variety of processes involving the modification and parsing of strings to use and change their data.

Slicing

- Slicing is the process of accessing a sub-sequence of a sequence by specifying a starting and ending index.

- **Using the List/array slicing [::] method**
- In Python, you perform slicing using the colon : operator.
- **Syntax:** sequence[start_index:end_index: *step*]
- where start_index is the index of the first element in the sub-sequence and end_index is the index of the last element in the sub-sequence (excluding the element at the end_index) and step: It is an optional argument that determines the increment between each index for slicing. Return Type: Returns a sliced object containing elements in the given range only.

```
arr[start:stop]          # items start through stop-1
arr[start:]          # items start through the rest of the array
arr[:stop]           # items from the beginning through stop-1
arr[:]               # a copy of the whole array
arr[start:stop:step]     # start through not past stop, by step
```

For Example

```
str="Hello World!"
print(str[:])
print(str[4])
print(str[1:4])
print(str[1:6:2])
print(str[::-1])
print(str[-5])
print(str[-8:-3])
```

```
Hello World!
o
ell
el
!dlroW olleH
o
o Wor
```

String Methods

Method name	Example	Purpose
title()	String.title()	returns the first character of each word is capitalized
lower()	String.lower()	Converts all uppercase characters in a string into lowercase characters and returns it.
upper()	String.upper()	Converts all lowercase characters in a string into uppercase characters and returns it.
count()	String.count(Substring, Start, End)	returns the number of times substring occurs in the given string. If start and end index is not specified then searching starts with 0 index and ends at the length of the string.
find()	String.find(Substring, Start, End)	returns the index of first occurrence of the substring in the given string (if found). If start and end index is not specified then searching starts with 0 index and ends at the length of the string. If the substring is not present in the given string then find() returns **-1**.

index()	String.index(Substring, Start, End)	Returns the index of a substring in the given string. If the substring is not found, it raises an Exception.
endswith()	String.endswith(Suffix, Start, End)	Returns True if the given string ends with the specified suffix otherwise returns False.
startswith()	String.startswith(Prefix , Start, End)	Returns True if the given starts with the specified prefix otherwise returns False.
isalnum()	String.isalnum()	Returns True if all characters in the string are alphanumeric
isalpha()	String.isalpha()	Returns True if all characters in the string are in the alphabet
isdigit()	String.isdigit()	Returns True if all characters in the string are digits
isspace()	String.isspace()	Returns True if string contains white spaces(tabs, spaces, newline, carriage return \n \t\r)
islower()	String.islower()	Returns True if string contains all lowercase characters
isupper()	String.isupper()	Returns True if string contains all uppercase characters
istitle()	String.istitle()	Returns True if string is titlecase i.e. first letter every word in the string in uppercase and rest in lowercase

strip()	String.strip(characters) : characters are optional	Removes any leading (starting) and trailing (ending) whitespaces from a given string.
lstrip()	String.lstrip(characters) : characters are optional	Removes any leading characters from given string
rstrip()	String.rstrip(characters) : characters are optional	Removes any trailing characters from given string
replace()	String.replace(old, new, count) count (optional) - the number of times you want to replace the old substring with the new string. If count is not specified, replace() method replaces all occurrences of the old substring with the new string	Replaces each matching occurrence of a substring with another string
join()	String.join(iterable)	Returns a string by joining all the elements of an iterable (list, string, tuple), separated by the given separator.
split()	String.split(separator)	Breaks down a string into a list of substrings using a chosen separator and returns it.
partition()	*string*.partition(*value*) : value:-The string to search for	It searches for a specified string, and splits the string into a tuple containing three elements. The first element contains the part before the specified string. The second element contains the specified string. The third element contains the part after the string.

```
str="hello hello World!"
print(str[:])
print(str[4])
print(str[1:4])
print(str[1:6:2])
print(str[::-1])
print(str[-5])
print(str[-8:-3])
print(str.title())
print(str.lower())
print(str.upper())
print(str.count('el'))
print(str.find('e'))
print(str.endswith('ld!'))
print(str.startswith('he'))
```

Output:

```
hello hello World!
o
ell
el
!dlroW olleh olleh
o
o Wor
Hello Hello World!
hello hello world!
HELLO HELLO WORLD!
2
1
True
True
```

List

- Data type list is an ordered sequence that is mutable and made up of one or more elements, may be of different types.
- Elements of a list are enclosed in square brackets and are separated by comma.
- A List is very useful to group together elements of mixed data types.
- Mutable: means changeble
- A List can have elements of different types such as integer, float, string, tuple or even other list.
- List indices starts with 0(Zero).
- A list may contain duplicate values.
- lists are ordered, it means that the elements have a defined order, and that order will not change.

Accessing elements from the List

- In order to access the list items refer to the index number. Use the **index operator [ ]** to access an item in a list. The index must be an integer. Nested lists are accessed using nested indexing.
- List items are indexed and they will be accessed by using index number:
- Index of first element is 0
- Negative indexing means start from the end. -1 refers to the last item, -2 refers to the second last item etc.

- **Range of Indexes**
- You can specify a range of indexes by specifying where to start and where to end the range.
- When specifying a range, the return value will be a new list with the specified items.

Creating a List in Python

- List is created by adding the elements in square brackets and are separated by comma.

```
# Blank List
l1=[]
print(l1)

#List: Demo of List with same type
l1=["c","c++","Java","Python"]
print(l1)

#List: Demo of List with duplicate values
l1=["c","c++","Java","C++","Python“]
print(l1)
```

```
#List: Demo of List with mixed type
l1=[101,"Ram", 75.70]
print(l1)

#List: Demo of nested List
l1=[[101,"c"],[102,"C++"],[103,"Java"]]
print(l1)
```

Sr. No.	Method	Syntax	Use
1	append()	list.append(element)	Adds an element at the end of the list
2	extend()	list1.append(list2 or sequence)	Add the elements of a list (or any iterable), to the end of the current list
3	insert()	list.insert(index, element	Adds an element at the specified position
4	pop()	list.pop()	Removes the element at the specified position. If index is not specified it return last element from the list
5	del	del(list[index])	Deletes one or more elements from the list.
6	remove()	list.remove(number)	Removes the item with the specified value
7	clear()	list.clear()	Removes all the elements from the list
8	reverse()	list.reverse()	Reverses the order of the list
9	copy()	list.copy()	Returns a copy of the list
10	count()	list.count(element)	Returns the number of elements with the specified value
11	index()	list.index(element)	Returns the index of the first element with the specified value
12	sort()	list.sort()	Sorts the list

	Method		Use
1	+	list1+list2	Combine two list
2	*	List1*count Ex. List1*2	Repeat all elements in the list as specific number of times.

Built in Functions

	Method	Syntax	Use
1	len()	Len(list)	Returns length of the list
2	max()	Max(list)	Returns maximum element from the list
3	min()	Min(list)	Returns minimum element from the list
4	sum()	Sum(list)	Returns sum of all elements of the list

Tuple